

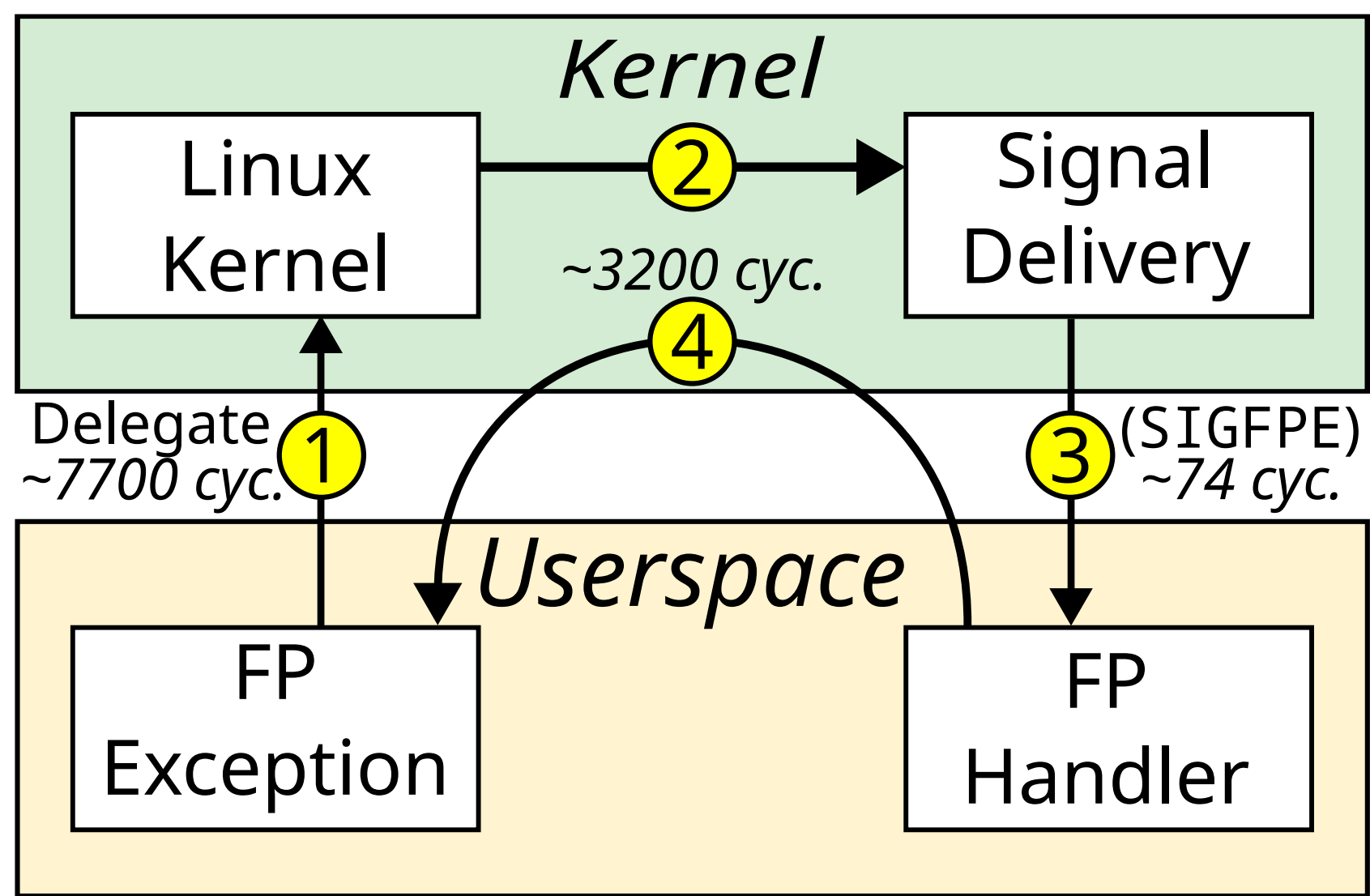
Hardware-based Kernel-Bypass Exceptions to Accelerate Floating Point Tracing and Virtualization

Karl Hallsby¹ Liam Strand¹ Peter Dinda¹

¹Northwestern University



How do you get an exception now?



Why Does this Fall Apart?

- Some tasks require significant numbers of exceptions: virtualization, tracing, etc.
- Floating point traps/exceptions quickly propagate and become frequent
- Kernels will never cause floating point traps, only user programs

Why are things this way?

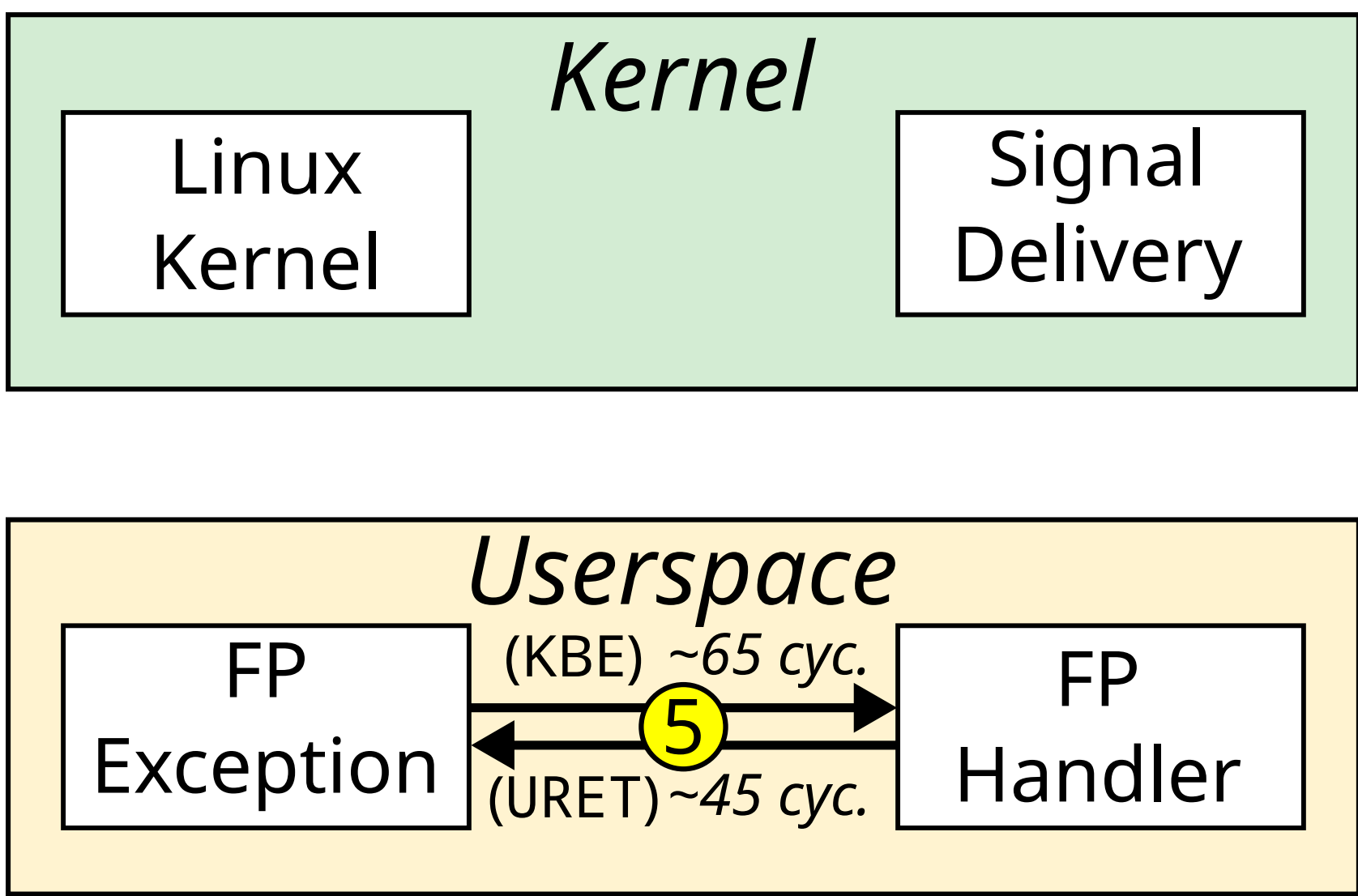
- Kernel does not trust user programs and retains all control
- 80/20 optimization: Exceptions are treated as rare

Can exceptions be made fast enough to serve as a form of control-flow?

What is RAFT-V? What changed?

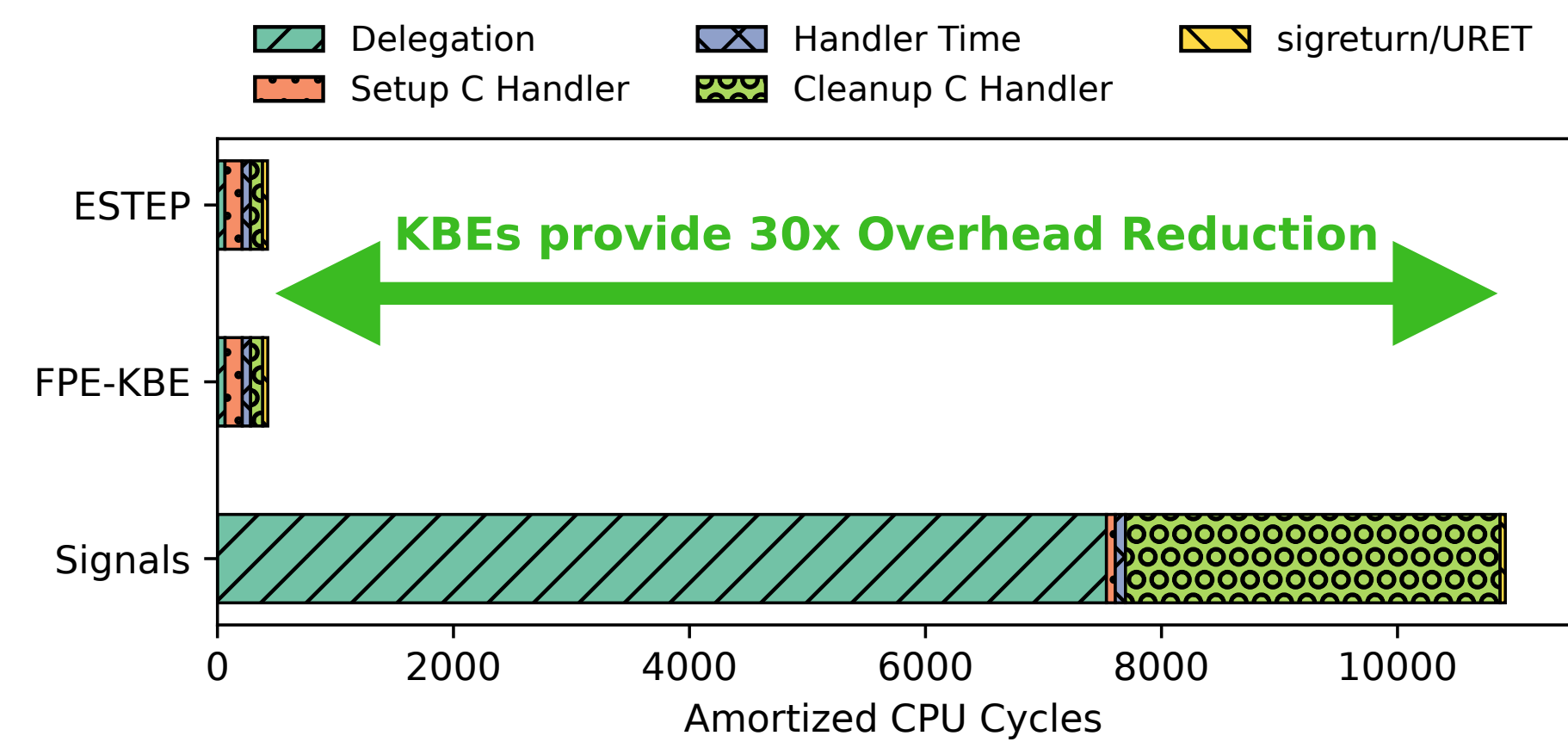
- Hardware can delegate exceptions directly to user-mode
- Software leverages this to enable **Kernel-Bypass Exceptions (KBEs)**, delivering exceptions **directly** to userspace
- Modified SonicBOOM [1] a modern high-performance out-of-order CPU, just 1–3% increase in resources
- Modified platform firmware (OpenSBI) to allow kernel to access hardware changes
- Modified Linux to cleanly handle HW state on context switches
- Linux module allowing user processes to subscribe to KBEs
- Evaluated in FireSim [2], a cycle-accurate FPGA-accelerated simulator, booting Linux

KBE Flow on RAFT-V

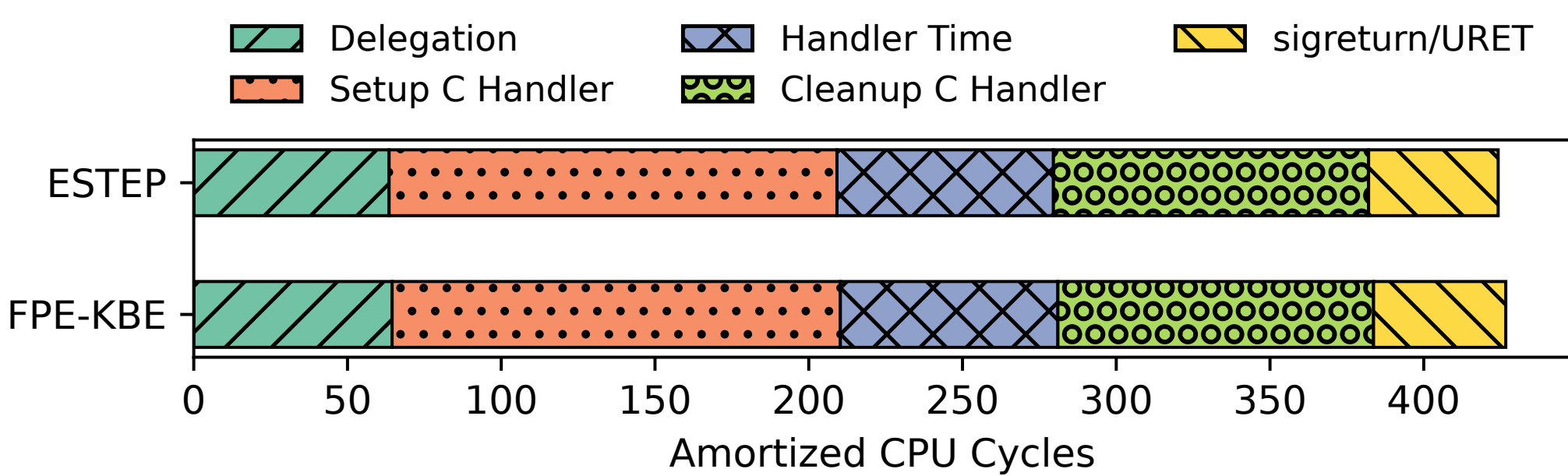


Kernel-bypass exceptions moves overheads and enables new features!

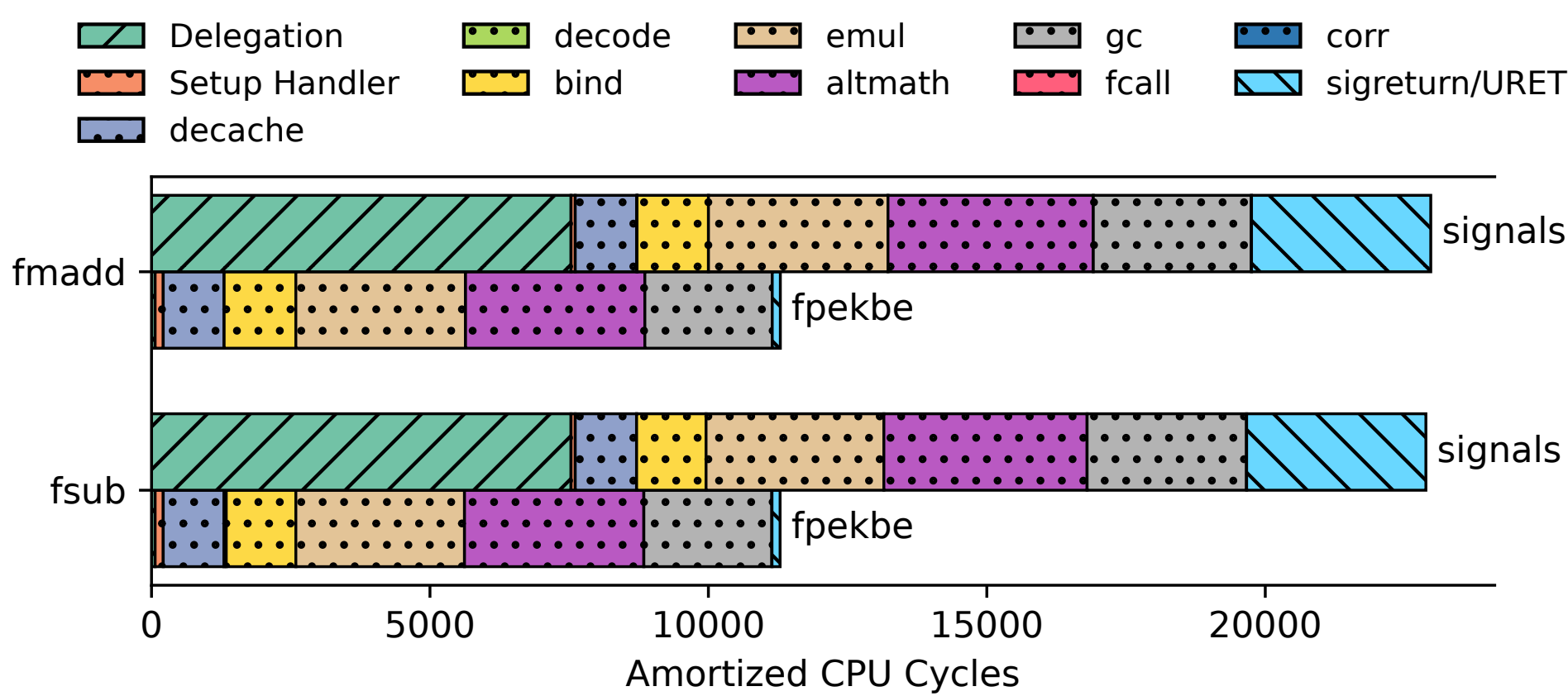
KBEs Provide up to 30× Speedup



Time Spent Moves to Software



Virtualization costs move to VM monitor



Each KBE removes 10k cycles of overhead

